ЛБ2

1. ТЕХНОЛОГІЧНА БАЗА (ОГЛЯД ІНСТРУМЕНТІВ, БІБЛІОТЕК І МОВ)

У проєкті базовою мовою взято Python 3. Він швидше та простіше працює з потрібними бібліотеками (OpenCV, MediaPipe, ONNX Runtime, PySide6), бо має зрілі офіційні обгортки та великий вибір готових прикладів. Це зменшує час на інтеграцію і дозволяє зосередитись на логіці мікрожестів та послідовностей. Критичні місця, де важлива мінімальна затримка, за потреби можна винести на GPU без зміни загальної архітектури.

Для комп'ютерного зору обрано OpenCV для роботи з камерою, перетворенням кольорів та базовим препроцесингом кадрів.

Для відстеження кисті використаємо MediaPipe Hands як швидку точку входу до 21 орієнтиру в реальному часі. Якщо буде потреба у власних моделях або альтернативних пайплайнах, передбачимо формат ONNX: інференс через ONNX Runtime з увімкненим GPU-бекендом (CUDA) на NVIDIA, із можливістю падати на CPU без змін коду.

Графічний інтерфейс та екранні підказки можна робити на PySide6/Qt. Це дозволить показувати індикатори режимів, стани команд та прості налаштування чутливості в одному вікні.

Інтеграцію з операційною системою на Windows буде реалізовано через стандартні виклики надсилання подій (гарячі клавіші, кліки, рух курсора).

2. ТЕОРЕТИЧНЕ ПОЯСНЕННЯ ЗАПРОПОНОВАНОГО РІШЕННЯ

2.1. Послідовний вигляд

У послідовному вигляді система працює як лінійний конвеєр «кадр – лендмарки – інтерпретація – команда». На вхід надходить відеопотік із камери, кадр приводиться до потрібного формату, після чого визначаються долоні та

орієнтир кисті. Отримані координати нормалізуються, на їх основі обчислюються ознаки рухів. Далі виконується інтерпретація жестів. За умови валідної послідовності формується команда для операційної системи, а на екрані оновлюються підказки та стани. Контекстну структуру рішення відображає IDEF0-діаграма, що наведено на рисунку 1.

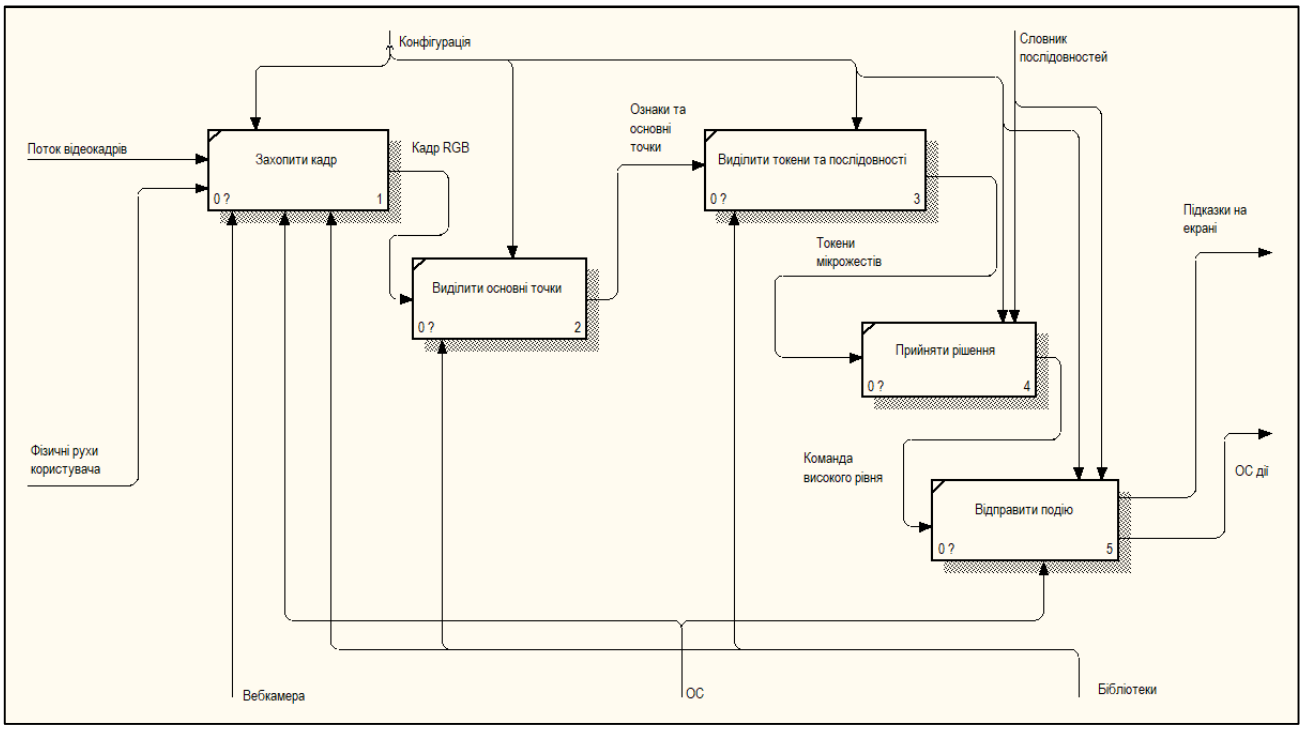

Рисунок 1 – Контекстна діаграма IDEF0

Внутрішня обробка подається лінійним ланцюгом від A1 до A5.

Блок A1 «Захопити кадр» приймає поток відеокадрів сцену через механізм вебкамеру і видає кадр у форматі RGB.

Блок A2 «Виділити основні точки» отримує цей кадр і визначає долоню та 21 орієнтир кисті для кожної знайденої руки разом із оцінкою впевненості.

Блок A3 «Виділити токени та послідовності» перетворює точки на ознаки рухів та виділяє базові дії.

Блок A4 «Прийняти рішення» застосовує словник послідовностей і правила підтвердження наміру. На виході формується команда високого рівня з урахуванням активного вікна ОС.

Блок A5 «Відправити подію» перетворює цю команду на конкретні дії операційної системи (комбінації клавіш, клацання, переміщення курсора), оновлює екранні підказки.

2.2. Паралельний вигляд

Паралельний варіант організовано як конвеєр з чотирьох ярусів. Захоплення кадру, виявлення лодонь, ухвалення рішення та інтеграція з ОС. Між ярусами стоять кільцеві буфери і черги. Вони дають незалежність обчислень та просту синхронізацію «виробник-споживач».

Ярус захвату читає кадри з камери і кладе їх у кільцевий буфер кадрів. Ярус виявлення лодонь бере останній кадр з буфера, виконує препроцесинг, детекцію долоні, обчислення точок та формує ознаки. Результат потрапляє у чергу дій.

Ярус команд читає з цієї черги, обробляє дії, формує намір та кладе команду у чергу команд.

Ярус UI OS витягує команди, надсилає події у систему і оновлює підказки на екрані.

Уся схема зображена на рисунку 2.

Рисунок 2 – Паралельно ярусна модель.

Обчислення у ярусі виявлення лодонь планується виконувати на GPU у моделі SIMT. Це скорочує час препроцесингу, детекції та точок.